1erParcial-t - Codigo Completo del Proyecto

REST + GraphQL + gRPC con Node.js, Express y MongoDB

=== package.json ===

```
{
  "name": "1erparcial-t",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "dev": "nodemon server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "@grpc/grpc-js": "^1.12.0",
    "@grpc/proto-loader": "^0.7.0",
    "express": "^5.2.1",
    "express-graphql": "^0.12.0",
    "graphql": "^15.10.3",
    "mongoose": "^9.10.2",
    "nodemon": "^3.1.14",
    "swagger-jsdoc": "^6.3.0",
    "swagger-ui-express": "^5.0.1"
  }
}
```

=== .env.example ===

```
PORT = 3000
MONGO_URI = mongodb://mongo:27017/empresa_db
```

=== .env ===

```
PORT = 3000
MONGO_URI = mongodb://mongo:27017/empresa_db
```

=== Dockerfile ===

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "run", "dev"]
```

=== docker-compose.yml ===

```
services:
  app:
    build: .
    container_name: Examen_parcial1
    ports:
    - "3000:3000"
    environment:
    - MONGO_URI=mongodb://mongo:27017/empresa_db
    depends_on:
    - mongo
  mongo:
    image: mongo:latest
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db

volumes:
  mongo-data:
```

=== DOCKER_GUIA.md ===

````
# GUIA DOCKER Y DOCKER COMPOSE

## 1. DOCKERFILE - Receta para construir tu app

El Dockerfile es como una receta de cocina que crea una imagen (una caja) con tu aplicacion.

### Linea por linea:

```
FROM node:20-alpine
```
Empieza desde la imagen base de Node.js 20 con Alpine Linux (es muy ligera, ~5MB).

```
WORKDIR /app
```
Crea la carpeta /app y se posiciona ahi. Es como hacer cd /app.

```
COPY package*.json ./
```
Copia package.json y package-lock.json al contenedor.

```
RUN npm install
```
Instala las dependencias. Esto se ejecuta UNA SOLA VEZ al construir la imagen.

```
COPY . .
```
Copia TODO el codigo (server.js, models/, etc.) al contenedor.
Lo hacemos despues de npm install para optimizar.
````

```
EXPOSE 3000
```
Documenta que la app usa el puerto 3000.

```
CMD ["npm", "run", "dev"]
```
Comando que se ejecuta cuando el contenedor ARRANCA.
RUN = al construir | CMD = al iniciar

---

2. DOCKER-COMPOSE.YML - Orquestador de contenedores

Compose levanta varios contenedores juntos (tu app + MongoDB) y los conecta.

Servicio 'app' (tu aplicacion Node.js):

```
app:
  build: .
```
Construye la imagen usando el Dockerfile de esta carpeta.

```
  container_name: Examen_parcial1
```
Nombre fijo del contenedor. Sin esto Docker pone nombres aleatorios.

```
  ports:
    - "3000:3000"
```
Mapeo: puerto_host:puerto_contenedor.
Tu computadora recibe localhost:3000 y lo redirige al contenedor.

```
  environment:
    - MONGO_URI=mongodb://mongo:27017/empresa_db
```
Variable de entorno. Importante: 'mongo' es el nombre del OTRO servicio.
Docker lo convierte automaticamente en IP interna.

```
  depends_on:
    - mongo
```
No inicies 'app' hasta que 'mongo' este creado.

Servicio 'mongo' (base de datos):

```
mongo:
  image: mongo:latest
```
Usa la imagen oficial de MongoDB desde Docker Hub. No necesita Dockerfile.

```
  ports:
    - "27017:27017"
```
Expone MongoDB para conectarte desde fuera (MongoDB Compass, terminal, etc.)

```
  volumes:
    - mongo-data:/data/db
```
Persistencia: guarda los datos en tu disco duro.
Sin esto, al apagar el contenedor se borra todo.

```
volumes:
  mongo-data:
```
Declara el volumen que Docker gestionara automaticamente.

---

3. COMANDOS DOCKER (Base)

```bash
docker images
```
Ver imagenes descargadas/construidas.

```bash
docker build -t nombre-imagen .
```
Construir imagen desde Dockerfile.

```bash
docker run -p 3000:3000 nombre-imagen
```
Correr un contenedor desde una imagen.

```bash
docker ps
```
Ver contenedores corriendo. Usar -a para ver todos.

```bash
docker stop <id>
docker rm <id>
docker rmi <nombre-imagen>
```

```
Parar, eliminar contenedor, eliminar imagen.

---

## 4. COMANDOS DOCKER COMPOSE (Los mas usados)

```bash
docker-compose up
```
Levantar todos los servicios. Muestra logs en terminal.

```bash
docker-compose up -d
```
Levantar en segundo plano (detached). Libera la terminal.

```bash
docker-compose up -d --build
```
Fuerza reconstruir la imagen. Usalo cuando cambies codigo.

```bash
docker-compose logs
```
Ver logs de todos los servicios.

```bash
docker-compose logs -f app
```
Ver logs de un servicio especifico y seguir escuchando (-f = follow).

```bash
docker-compose down
```
Apagar y eliminar contenedores. Los volumenes se mantienen.

```bash
docker-compose down -v
```
Apagar TODO incluyendo volumenes. CUIDADO: borra la base de datos.

```bash
docker-compose ps
```
Ver estado de los contenedores de este compose.

```bash
docker-compose exec app sh
```
Abrir terminal dentro del contenedor 'app' para debuguear.

---

5. DIAGRAMA DE RED

```
TU COMPUTADORA (Host)
  localhost:3000  -->  Contenedor 'app' (Node.js)
  localhost:27017 -->  Contenedor 'mongo' (MongoDB)

Red interna Docker:
  app <---> mongo (se comunican por nombre 'mongo')

Volumen:
  mongo-data:/data/db (persistencia en disco)
```

---

6. DIFERENCIA CLAVE: CMD vs RUN

Comando	Cuando se ejecuta	Ejemplo
RUN	Al CONSTRUIR la imagen	npm install
CMD	Al INICIAR el contenedor	npm run dev

RUN crea capas en la imagen. CMD es el punto de entrada del contenedor.
```

=== models/Trabajador.js ===

```
const mongoose = require('mongoose');

const trabajadorSchema = new mongoose.Schema({
    nombre: {type:String, required:true},
    apellido: {type:String, required:true},
    cedula: {type:String, required:true},
    cargo: {type:String, required:true},
    departamento: {type:String, required:true},
    fechaIngreso: {type:Date, required:true},
});

module.exports = mongoose.model('Trabajador', trabajadorSchema);
```

=== server.js ===

```
const express = require('express');
const mongoose = require('mongoose');
const Trabajador = require('./models/trabajador');

//parte graphql

const { graphqlHTTP } = require('express-graphql');
const { buildSchema } = require('graphql');
```

```
// parte swagger
const swaggerJsdoc = require('swagger-jsdoc');
const swaggerUi = require('swagger-ui-express');

const app = express();
const PORT = process.env.PORT || 3000;
const MONGO_URI = process.env.MONGO_URI || 'mongodb://localhost:27017/empresa_db';

app.use(express.json());

mongoose.connect(MONGO_URI)
    .then (() => {
        console.log('Connected to MongoDB');
    })

// ========================================
// CONFIGURACION SWAGGER (Documentacion API)
// ========================================
// swagger-jsdoc lee estos comentarios @swagger y genera un JSON OpenAPI

const swaggerOptions = {
    definition: {
        openapi: '3.0.0',
        info: {
            title: 'API Trabajadores',
            version: '1.0.0',
            description: 'API REST + GraphQL para gestionar trabajadores con MongoDB',
        },
        servers: [
            {
                url: `http://localhost:${PORT}`,
                description: 'Servidor local',
            },
        ],
    },
    // Archivos donde swagger buscara comentarios @swagger
    apis: ['./server.js'],
};

const specs = swaggerJsdoc(swaggerOptions);

// Ruta donde se mostrara la documentacion interactiva
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(specs));

// ========================================
// ENDPOINTS REST (con documentacion Swagger)
// ========================================

/**
 * @swagger
 * /trabajador:
 *   get:
 *     summary: Obtener todos los trabajadores
 *     description: Retorna una lista con todos los trabajadores registrados en MongoDB
 *     tags: [Trabajadores]
 *     responses:
 *       200:
 *         description: Lista de trabajadores
 *         content:
 *           application/json:
 *             schema:
 *               type: array
 *               items:
 *                 type: object
 *                 properties:
 *                   _id:
 *                     type: string
 *                     example: "64b8f..."
 *                   nombre:
 *                     type: string
 *                     example: "Juan"
 *                   apellido:
 *                     type: string
 *                     example: "Perez"
 *                   cedula:
 *                     type: string
 *                     example: "V12345678"
 *                   cargo:
 *                     type: string
 *                     example: "Desarrollador"
 *                   departamento:
 *                     type: string
 *                     example: "Tecnologia"
 *                   fechaIngreso:
 *                     type: string
 *                     example: "2025-07-20"
 */
app.get('/trabajador', async (req, res) => {
    res.json(await Trabajador.find());
});

/**
 * @swagger
 * /trabajador:
 *   post:
 *     summary: Crear un nuevo trabajador
 *     description: Registra un trabajador en la base de datos
 *     tags: [Trabajadores]
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             required: [nombre, apellido, cedula, cargo, departamento, fechaIngreso]
```

```
 *             properties:
 *               nombre:
 *                 type: string
 *                 example: "Carlos"
 *               apellido:
 *                 type: string
 *                 example: "Ramirez"
 *               cedula:
 *                 type: string
 *                 example: "V-9876543"
 *               cargo:
 *                 type: string
 *                 example: "Ingeniero DevOps"
 *               departamento:
 *                 type: string
 *                 example: "Infraestructura"
 *               fechaIngreso:
 *                 type: string
 *                 example: "2025-07-20"
 *     responses:
 *       201:
 *         description: Trabajador creado exitosamente
 *       400:
 *         description: Error de validacion (faltan campos requeridos)
 */
app.post('/trabajador', async (req, res) => {
    res.status(201).json(await Trabajador.create(req.body));
});

/**
 * @swagger
 * /trabajador/{id}:
 *   put:
 *     summary: Actualizar un trabajador
 *     description: Modifica los datos de un trabajador existente por su ID
 *     tags: [Trabajadores]
 *     parameters:
 *       - in: path
 *         name: id
 *         required: true
 *         schema:
 *           type: string
 *         description: ID del trabajador en MongoDB
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             properties:
 *               nombre:
 *                 type: string
 *               apellido:
 *                 type: string
 *               cedula:
 *                 type: string
 *               cargo:
 *                 type: string
 *               departamento:
 *                 type: string
 *               fechaIngreso:
 *                 type: string
 *     responses:
 *       200:
 *         description: Trabajador actualizado
 *       404:
 *         description: Trabajador no encontrado
 */
app.put('/trabajador/:id', async (req, res) => {
    const item = await Trabajador.findByIdAndUpdate(req.params.id, req.body, {new: true});
    item ? res.status(200).json(item) : res.status(404).json({message: 'trabajador not found'});
});

/**
 * @swagger
 * /trabajador/{id}:
 *   delete:
 *     summary: Eliminar un trabajador
 *     description: Borra un trabajador de la base de datos por su ID
 *     tags: [Trabajadores]
 *     parameters:
 *       - in: path
 *         name: id
 *         required: true
 *         schema:
 *           type: string
 *         description: ID del trabajador en MongoDB
 *     responses:
 *       200:
 *         description: Trabajador eliminado
 *         content:
 *           application/json:
 *             schema:
 *               type: object
 *               properties:
 *                 message:
 *                   type: string
 *                   example: "trabajador deleted"
 *       404:
 *         description: Trabajador no encontrado
 */
app.delete('/trabajador/:id', async (req, res) => {
    const item = await Trabajador.findByIdAndDelete(req.params.id);
    item ? res.status(200).json({message: 'trabajador deleted'}) : res.status(404).json({message: 'trabajador not found'
});
```

```
})

//graphql
const schema = buildSchema(
    `
    type Trabajador {
        _id: ID
        nombre: String
        apellido: String
        cedula: String
        cargo: String
        departamento: String
        fechaIngreso: String
    }
    input TrabajadorInput {
        nombre: String
        apellido: String
        cedula: String
        cargo: String
        departamento: String
        fechaIngreso: String
    }
    input TrabajadorUpdateInput {
        nombre: String
        apellido: String
        cedula: String
        cargo: String
        departamento: String
        fechaIngreso: String
    }

    type Query {
        obtenerTrabajadores: [Trabajador]
        obtenerTrabajador(_id: ID): Trabajador
    }
    type Mutation {
        crearTrabajador(input: TrabajadorInput): Trabajador
        actualizarTrabajador(_id: ID, input: TrabajadorUpdateInput): Trabajador
        eliminarTrabajador(_id: ID): String
    }
    `
);
const root = {
    obtenerTrabajadores: async () => {
        return await Trabajador.find();
    },
    obtenerTrabajador: async ({_id}) => {
        return await Trabajador.findById(_id);
    },
    crearTrabajador: async ({input}) => {
        const trabajador = new Trabajador(input);
        await trabajador.save();
        return trabajador;
    },
    actualizarTrabajador: async ({_id, input}) => {
        const trabajador = await Trabajador.findByIdAndUpdate(_id, input, {new: true});
        if (!trabajador) {
            throw new Error('Trabajador no encontrado');
        }
        return trabajador;
    },
    eliminarTrabajador: async ({_id}) => {
        await Trabajador.findByIdAndDelete(_id);
        return 'Trabajador eliminado';
    }

};

app.use('/graphql', graphqlHTTP({
    schema: schema,
    rootValue: root,
    graphiql: true
}));

app.listen(PORT,() => {
    console.log('Servidor corriendo en puerto: ' + PORT);
    console.log('Documentacion Swagger disponible en: http://localhost:' + PORT + '/api-docs');
    console.log('GraphQL Playground disponible en: http://localhost:' + PORT + '/graphql');
});
```

=== grpc-ejemplo/trabajador.proto ===

```
syntax = "proto3";

// Este archivo define el CONTRATO entre cliente y servidor
// Es como un "idioma común" que ambos deben entender

service TrabajadorService {
  // Listar todos los trabajadores (no necesita argumentos)
  rpc Listar (Vacio) returns (Lista);

  // Obtener un trabajador por su ID de MongoDB
  rpc Obtener (Id) returns (Trabajador);

  // Crear un nuevo trabajador
  rpc Crear (TrabajadorInput) returns (Trabajador);

  // Actualizar un trabajador existente
  rpc Actualizar (TrabajadorActualizar) returns (Trabajador);

  // Eliminar un trabajador por ID
  rpc Borrar (Id) returns (Vacio);
}

// Mensaje vacio: se usa cuando una funcion no necesita datos de entrada
```

```
message Vacio {}

// Mensaje para enviar/recibir un ID
message Id {
  string id = 1;  // El numero 1 es el "tag" del campo en protobuf
}

// Mensaje que representa un Trabajador completo (lo que devolvemos)
message Trabajador {
  string id = 1;           // _id de MongoDB (convertido a string)
  string nombre = 2;
  string apellido = 3;
  string cedula = 4;
  string cargo = 5;
  string departamento = 6;
  string fechaIngreso = 7; // Fecha como string formato ISO (ej: "2025-07-20")
}

// Mensaje para crear un nuevo trabajador (sin ID, Mongo lo genera)
message TrabajadorInput {
  string nombre = 1;
  string apellido = 2;
  string cedula = 3;
  string cargo = 4;
  string departamento = 5;
  string fechaIngreso = 6;
}

// Mensaje para actualizar: necesita ID + datos nuevos
message TrabajadorActualizar {
  string id = 1;
  TrabajadorInput datos = 2;
}

// Mensaje que contiene una lista de trabajadores
message Lista {
  repeated Trabajador items = 1;  // "repeated" = array/lista
}
```

=== grpc-ejemplo/grpc-server.js ===

```
const grpc = require('@grpc/grpc-js');
const protoLoader = require('@grpc/proto-loader');
const mongoose = require('mongoose');

// ============================================
// 1. CONEXION A MONGODB
// ============================================
// Nos conectamos a la misma base de datos que usa el proyecto principal
// Si usas Docker: mongodb://mongodb:27017/empresa_db
// Si es local:    mongodb://localhost:27017/empresa_db

const MONGO_URI = process.env.MONGO_URI || 'mongodb://localhost:27017/empresa_db';

mongoose.connect(MONGO_URI)
  .then(() => console.log('[gRPC Server] Conectado a MongoDB'))
  .catch(err => console.error('[gRPC Server] Error MongoDB:', err));

// ============================================
// 2. DEFINIR MODELO (igual que en tu proyecto)
// ============================================
const trabajadorSchema = new mongoose.Schema({
  nombre: { type: String, required: true },
  apellido: { type: String, required: true },
  cedula: { type: String, required: true },
  cargo: { type: String, required: true },
  departamento: { type: String, required: true },
  fechaIngreso: { type: Date, required: true },
});

const Trabajador = mongoose.model('Trabajador', trabajadorSchema);

// ============================================
// 3. CARGAR EL ARCHIVO .PROTO
// ============================================
// protoLoader convierte el archivo .proto en un objeto JavaScript usable

const pkgDef = protoLoader.loadSync(__dirname + '/trabajador.proto', {
  keepCase: true,
  longs: String,
  enums: String,
  defaults: true,
  oneofs: true
});

const proto = grpc.loadPackageDefinition(pkgDef).trabajador;

// ============================================
// 4. IMPLEMENTAR LOS METODOS DEL SERVICIO
// ============================================
// Cada funcion aqui responde a un "rpc" definido en el .proto
// call = objeto con los datos enviados por el cliente
// callback = funcion que usamos para responder al cliente

const server = new grpc.Server();

server.addService(proto.TrabajadorService.service, {

  // LISTAR TODOS
  Listar: async (_, callback) => {
    try {
      const data = await Trabajador.find();

      // Convertir cada documento MongoDB al formato del .proto
      const items = data.map(doc => ({
```

```
      id: doc._id.toString(),
      nombre: doc.nombre,
      apellido: doc.apellido,
      cedula: doc.cedula,
      cargo: doc.cargo,
      departamento: doc.departamento,
      fechaIngreso: doc.fechaIngreso.toISOString().split('T')[0], // "2025-07-20"
    }));

    callback(null, { items });
  } catch (err) {
    callback({ code: grpc.status.INTERNAL, message: err.message });
  }
},

// OBTENER UNO POR ID
Obtener: async (call, callback) => {
  try {
    const doc = await Trabajador.findById(call.request.id);
    if (!doc) {
      return callback({ code: grpc.status.NOT_FOUND, message: 'Trabajador no encontrado' });
    }

    callback(null, {
      id: doc._id.toString(),
      nombre: doc.nombre,
      apellido: doc.apellido,
      cedula: doc.cedula,
      cargo: doc.cargo,
      departamento: doc.departamento,
      fechaIngreso: doc.fechaIngreso.toISOString().split('T')[0],
    });
  } catch (err) {
    callback({ code: grpc.status.INTERNAL, message: err.message });
  }
},

// CREAR NUEVO
Crear: async (call, callback) => {
  try {
    const { nombre, apellido, cedula, cargo, departamento, fechaIngreso } = call.request;

    const doc = await Trabajador.create({
      nombre,
      apellido,
      cedula,
      cargo,
      departamento,
      fechaIngreso: new Date(fechaIngreso), // Convertir string a Date de MongoDB
    });

    callback(null, {
      id: doc._id.toString(),
      nombre: doc.nombre,
      apellido: doc.apellido,
      cedula: doc.cedula,
      cargo: doc.cargo,
      departamento: doc.departamento,
      fechaIngreso: doc.fechaIngreso.toISOString().split('T')[0],
    });
  } catch (err) {
    callback({ code: grpc.status.INTERNAL, message: err.message });
  }
},

// ACTUALIZAR
Actualizar: async (call, callback) => {
  try {
    const { id, datos } = call.request;
    const updateData = { ...datos };

    // Si viene fechaIngreso, convertirla a Date
    if (updateData.fechaIngreso) {
      updateData.fechaIngreso = new Date(updateData.fechaIngreso);
    }

    const doc = await Trabajador.findByIdAndUpdate(id, updateData, { new: true });
    if (!doc) {
      return callback({ code: grpc.status.NOT_FOUND, message: 'Trabajador no encontrado' });
    }

    callback(null, {
      id: doc._id.toString(),
      nombre: doc.nombre,
      apellido: doc.apellido,
      cedula: doc.cedula,
      cargo: doc.cargo,
      departamento: doc.departamento,
      fechaIngreso: doc.fechaIngreso.toISOString().split('T')[0],
    });
  } catch (err) {
    callback({ code: grpc.status.INTERNAL, message: err.message });
  }
},

// BORRAR
Borrar: async (call, callback) => {
  try {
    const doc = await Trabajador.findByIdAndDelete(call.request.id);
    if (!doc) {
      return callback({ code: grpc.status.NOT_FOUND, message: 'Trabajador no encontrado' });
    }
    callback(null, {});
  } catch (err) {
    callback({ code: grpc.status.INTERNAL, message: err.message });
```

```
    }
  }

});

// ============================================
// 5. ARRANCAR EL SERVIDOR
// ============================================

const PORT = '0.0.0.0:50051';
server.bindAsync(PORT, grpc.ServerCredentials.createInsecure(), (err) => {
  if (err) {
    console.error('Error iniciando servidor:', err);
    return;
  }
  console.log(`[gRPC Server] Escuchando en ${PORT}`);
  server.start();
});
```

=== grpc-ejemplo/grpc-client.js ===

```
const grpc = require('@grpc/grpc-js');
const protoLoader = require('@grpc/proto-loader');

// ============================================
// CLIENTE gRPC - PARA PROBAR EL SERVIDOR
// ============================================
// Este archivo se conecta al servidor gRPC y ejecuta operaciones de prueba

// 1. Cargar el archivo .proto (mismo archivo que usa el servidor)
const pkgDef = protoLoader.loadSync(__dirname + '/trabajador.proto', {
  keepCase: true,
  longs: String,
  enums: String,
  defaults: true,
  oneofs: true
});

const proto = grpc.loadPackageDefinition(pkgDef).trabajador;

// 2. Crear el cliente apuntando al servidor
const client = new proto.TrabajadorService(
  'localhost:50051',
  grpc.credentials.createInsecure()
);

// ============================================
// FUNCION AUXILIAR: Promisificar las llamadas
// ============================================
// gRPC usa callbacks, pero con Promesas el codigo es mas limpio

function llamar(metodo, datos) {
  return new Promise((resolve, reject) => {
    client[metodo](datos, (err, respuesta) => {
      if (err) reject(err);
      else resolve(respuesta);
    });
  });
}

// ============================================
// PRUEBAS: Ejecutar operaciones una tras otra
// ============================================

async function probar() {
  try {
    console.log('=== [CLIENTE gRPC] Iniciando pruebas ===\n');

    // 1. CREAR un trabajador
    console.log('1. Creando trabajador...');
    const nuevo = await llamar('Crear', {
      nombre: 'Carlos',
      apellido: 'Ramirez',
      cedula: 'V-9876543',
      cargo: 'Ingeniero DevOps',
      departamento: 'Infraestructura',
      fechaIngreso: '2025-07-20'
    });
    console.log('   Creado:', nuevo);
    const idCreado = nuevo.id;

    // 2. LISTAR todos los trabajadores
    console.log('\n2. Listando trabajadores...');
    const lista = await llamar('Listar', {});
    console.log('   Total:', lista.items.length);
    lista.items.forEach(t => {
      console.log(`   - ${t.nombre} ${t.apellido} | ${t.cargo} | ${t.departamento}`);
    });

    // 3. OBTENER el trabajador que acabamos de crear
    console.log('\n3. Obteniendo trabajador por ID...');
    const uno = await llamar('Obtener', { id: idCreado });
    console.log('   Encontrado:', uno.nombre, uno.apellido, '- Cédula:', uno.cedula);

    // 4. ACTUALIZAR el cargo
    console.log('\n4. Actualizando cargo...');
    const actualizado = await llamar('Actualizar', {
      id: idCreado,
      datos: {
        nombre: 'Carlos',
        apellido: 'Ramirez',
        cedula: 'V-9876543',
        cargo: 'Lider DevOps',  // <-- Cambiamos el cargo
        departamento: 'Infraestructura',
        fechaIngreso: '2025-07-20'
```

```
      }
    });
    console.log('   Nuevo cargo:', actualizado.cargo);

    // 5. BORRAR el trabajador
    console.log('\n5. Borrando trabajador...');
    await llamar('Borrar', { id: idCreado });
    console.log('   Borrado exitosamente');

    // 6. VERIFICAR que se borro (listar de nuevo)
    console.log('\n6. Verificando despues de borrar...');
    const listaFinal = await llamar('Listar', {});
    console.log('   Total ahora:', listaFinal.items.length);

    console.log('\n=== [CLIENTE gRPC] Pruebas completadas ===');
    process.exit(0);

  } catch (err) {
    console.error('\n[ERROR]', err.message);
    process.exit(1);
  }
}

// Esperar 1 segundo para asegurar que el servidor esta listo
setTimeout(probar, 1000);
```

=== grpc-ejemplo/README.md ===

```
# Ejemplo gRPC - Trabajador

Este es un ejemplo minimo de gRPC con Node.js y MongoDB usando el modelo completo de Trabajador.

## Archivos

| Archivo | Descripcion |
|---------|-------------|
| `trabajador.proto` | Contrato del servicio (define que metodos existen) |
| `grpc-server.js` | Servidor gRPC que conecta a MongoDB |
| `grpc-client.js` | Cliente de prueba que ejecuta todas las operaciones |

## Instalacion

```bash
npm install @grpc/grpc-js @grpc/proto-loader mongoose
```

O si estas en la carpeta raiz del proyecto (1erParcial-t):

```bash
npm install @grpc/grpc-js @grpc/proto-loader
```

Ejecucion

Paso 1: Asegurar que MongoDB este corriendo

Si usas Docker Compose del proyecto principal:
```bash
docker-compose up -d mongodb
```

Si es MongoDB local:
```bash
mongod
```

Paso 2: Iniciar el servidor gRPC

```bash
node grpc-ejemplo/grpc-server.js
```

Deberias ver:
```
[gRPC Server] Conectado a MongoDB
[gRPC Server] Escuchando en 0.0.0.0:50051
```

Paso 3: Ejecutar el cliente de prueba

En otra terminal:

```bash
node grpc-ejemplo/grpc-client.js
```

El cliente ejecutara automaticamente:
1. Crear un trabajador
2. Listar todos
3. Obtener uno por ID
4. Actualizarlo
5. Borrarlo
6. Verificar el borrado

Diferencias clave: REST vs GraphQL vs gRPC

Caracteristica	REST	GraphQL	gRPC
Protocolo	HTTP/1.1	HTTP/1.1	HTTP/2
Formato	JSON	JSON	Protobuf (binario)
Velocidad	Media	Media	Muy rapida
Uso ideal	APIs publicas	Frontend flexible	Microservicios internos
Contrato	OpenAPI/Swagger	Schema SDL	Archivo .proto

Puerto del servidor
```

```
- gRPC Server: **50051** (puerto por defecto de gRPC)
- REST API: **3000** (del proyecto principal)
- GraphQL: **3000/graphql** (del proyecto principal)
```

=== README.md ===

```
# Proyecto 1erParcial-t - API Trabajadores

Proyecto completo de microservicios con Node.js, Express, MongoDB, GraphQL, gRPC y Swagger.

---

## Tecnologias incluidas

| Tecnologia | Uso | Ruta |
|------------|-----|------|
| **Express** | Framework web | Base del servidor |
| **MongoDB + Mongoose** | Base de datos | `mongodb://localhost:27017/empresa_db` |
| **REST API** | Endpoints HTTP tradicionales | `/trabajador` |
| **GraphQL** | API flexible con queries y mutations | `/graphql` |
| **Swagger/OpenAPI** | Documentacion interactiva de la API REST | `/api-docs` |
| **gRPC** | Comunicacion rapida entre microservicios | Puerto `50051` |
| **Docker** | Contenedores para Node.js y MongoDB | `docker-compose.yml` |

---

## Instalacion

```bash
npm install
```

Ejecucion

Opcion 1: Con Docker (recomendado)

```bash
docker-compose up -d
```

Levanta MongoDB + Node.js automaticamente.

Opcion 2: Local (necesitas MongoDB corriendo)

```bash
# Terminal 1: Asegurar MongoDB
docker-compose up -d mongodb
# o mongod si lo tienes instalado localmente

# Terminal 2: Servidor principal (REST + GraphQL + Swagger)
npm run dev
# o: node server.js

# Terminal 3: Servidor gRPC (opcional)
node grpc-ejemplo/grpc-server.js
```

Endpoints y URLs

Servicio	URL	Descripcion
REST API - GET	`http://localhost:3000/trabajador`	Listar trabajadores
REST API - POST	`http://localhost:3000/trabajador`	Crear trabajador
REST API - PUT	`http://localhost:3000/trabajador/:id`	Actualizar trabajador
REST API - DELETE	`http://localhost:3000/trabajador/:id`	Eliminar trabajador
Swagger UI	**`http://localhost:3000/api-docs`**	**Documentacion interactiva**
GraphQL	`http://localhost:3000/graphql`	Playground de GraphQL
gRPC Server	`localhost:50051`	Servicio gRPC (sin navegador)

Swagger - Documentacion Interactiva

Que es Swagger?

Swagger (ahora llamado OpenAPI) es una herramienta que genera documentacion automatica de tu API REST. A partir de comen
tarios especiales en el codigo (`@swagger`), crea una pagina web donde puedes:

- **Ver todos los endpoints** con sus parametros y respuestas
- **Probar los endpoints** directamente desde el navegador (sin Postman)
- **Ver ejemplos** de como enviar y recibir datos
- **Descargar** la especificacion OpenAPI en JSON

Como funciona en este proyecto?

En `server.js`, arriba de cada endpoint REST hay un bloque de comentios como este:

```javascript
/**
 * @swagger
 * /trabajador:
 *   get:
 *     summary: Obtener todos los trabajadores
 *     ...
 */
app.get('/trabajador', ...)
```

La libreria `swagger-jsdoc` lee estos comentarios al iniciar el servidor y genera un archivo JSON OpenAPI. Luego `swagge
r-ui-express` muestra ese JSON como una pagina web bonita en `/api-docs`.
```

````
### Como usarlo?

1. Inicia el servidor: `node server.js`
2. Abre tu navegador en: **`http://localhost:3000/api-docs`**
3. Veras una pagina con todos los endpoints documentados
4. Haz clic en cualquier endpoint (ej: `POST /trabajador`)
5. Presiona "Try it out"
6. Edita el JSON de ejemplo y presiona "Execute"
7. Veras la respuesta del servidor abajo

### Ventajas de usar Swagger

- **No necesitas Postman**: Pruebas directamente desde el navegador
- **Siempre actualizado**: La documentacion se genera del codigo fuente
- **Claro para frontend**: Los desarrolladores del frontend ven exactamente que enviar
- **Estandar industrial**: OpenAPI es el estandar mundial de documentacion de APIs

---

## GraphQL

Accede al playground en `http://localhost:3000/graphql`

### Queries

```graphql
query {
  obtenerTrabajadores {
    _id
    nombre
    apellido
    cargo
  }
}
```

```graphql
query {
  obtenerTrabajador(_id: "64b8f...") {
    nombre
    cedula
    departamento
  }
}
```

### Mutations

```graphql
mutation {
  crearTrabajador(input: {
    nombre: "Carlos"
    apellido: "Ramirez"
    cedula: "V-9876543"
    cargo: "DevOps"
    departamento: "Infraestructura"
    fechaIngreso: "2025-07-20"
  }) {
    _id
    nombre
  }
}
```

```graphql
mutation {
  actualizarTrabajador(_id: "64b8f...", input: {
    cargo: "Lider DevOps"
  }) {
    nombre
    cargo
  }
}
```

```graphql
mutation {
  eliminarTrabajador(_id: "64b8f...")
}
```

---

## gRPC

Ver carpeta `grpc-ejemplo/` para el ejemplo completo.

```bash
# Terminal 1: Servidor gRPC
node grpc-ejemplo/grpc-server.js

# Terminal 2: Cliente de prueba
node grpc-ejemplo/grpc-client.js
```

---

## Estructura de archivos

```
1erParcial-t/
 server.js                # Servidor principal (REST + GraphQL + Swagger)
 models/
    Trabajador.js         # Modelo Mongoose
````
````

```
 grpc-ejemplo/             # Ejemplo gRPC
    trabajador.proto      # Contrato gRPC
    grpc-server.js        # Servidor gRPC
    grpc-client.js        # Cliente de prueba
    README.md             # Guia gRPC
 package.json              # Dependencias
 docker-compose.yml        # Docker (Node + MongoDB)
 Dockerfile                # Imagen de Node.js
 .env                      # Variables de entorno
 .env.example              # Ejemplo de variables
 README.md                 # Este archivo
 CODIGO_COMPLETO.pdf       # Todo el codigo en PDF
```

Modelo de datos (Trabajador)

Campo	Tipo	Requerido	Ejemplo
nombre	String	Si	"Juan"
apellido	String	Si	"Perez"
cedula	String	Si	"V-12345678"
cargo	String	Si	"Desarrollador"
departamento	String	Si	"Tecnologia"
fechaIngreso	Date	Si	"2025-07-20"

Notas importantes

- **Formato de fecha**: Usar `YYYY-MM-DD` (ej: `2025-07-20`). MongoDB lo guarda como `Date`.
- **ID en GraphQL**: Usar `_id` (con guion bajo) para coincidir con MongoDB.
- **ID en REST**: Usar `id` en la URL (ej: `/trabajador/64b8f...`).
- **Swagger**: No documenta GraphQL ni gRPC, solo la parte REST.

Comandos utiles

```bash
# Instalar dependencias
npm install

# Iniciar en desarrollo (con nodemon)
npm run dev

# Iniciar en produccion
node server.js

# Generar PDF con todo el codigo
python3 generar_pdf.py

# Docker
 docker-compose up -d        # Levantar todo
 docker-compose down         # Bajar todo
 docker-compose logs -f app  # Ver logs de Node.js
```